

Arrays in Java

14 August 2026 05:55

Java arrays are objects that store multiple elements of the same data type in a single contiguous memory block.

Arrays allow us to group similar data together and access them using an index.

Arrays in Java

Arrays in Java are index based, meaning the first element is at index 0 and the second at index 1 and so on. In Java an array is considered an object of dynamically created class. All arrays in Java:

- Derived from the Object class.
- Array can store primitive values or objects and Java supports both single dimensional and multi-dimensional array.

Single-dimensional array in Java

A single dimensional array in Java is a linear collection of elements of the same data type. It can be declared and instantiated using the following syntax:

```
datatype[] arr; or
datatype []arr; or
datatype arr[];
```

Syntax of array instantiation:

```
Arr = new datatype[size];
```

The **new** keyword allocates the memory dynamically(at runtime) on the heap memory area.

```
package MyArrays;
import java.util.*;
```

```
/**
 * @author (your name)
 * @version (a version number or a date)
 */
public class SDArrayDemo
{
    public static Scanner sc = new Scanner(System.in);
    public static void main(String[] args){
        System.out.print("\n");
        int[] Arr1, i; //all the variable will become an array
        int []Arr2, j; //all the variable will become an array
        int Arr3[], k; //all the variables will not become an array
        Arr1 = new int[5];
        Arr2 = new int[5];
        Arr3 = new int[5];

        // i = 0; // this type of assignment is not allowed in the array
        // j = 0; //j is an array
```

```

    k = 0;
    //Declaring and allocating memory for the array.
    int Arr4[] = new int[5];

    Arr1[0] = 10;
    Arr1[1] = 20;
    Arr1[2] = 30;
    Arr1[3] = 40;
    Arr1[4] = 50;
    System.out.print("\nArr1[");
    for(k = 0; k < Arr1.length - 1; k++){
        System.out.print(Arr1[k]+", ");
    }
    System.out.print(Arr1[k]+"]\n");

    String []month = {"Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul",
        "Aug", "Sep", "Oct", "Nov", "Dec"};
    System.out.print("\nMonths: ");
    for(String m : month){ //for each loop is mainly used for Arrays
and Collection
        System.out.print(m+" ");
    }
    System.out.print("\n");

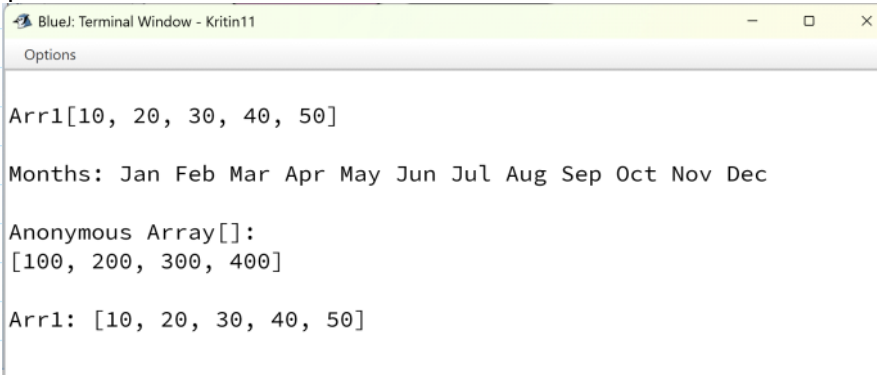
    SDArrayUtility.printSDArray(new int[]{100,200,300,400}, "Anonymous
Array[]: ");

    //Don't use in school examination
    System.out.print("\nArr1: " + java.util.Arrays.toString(Arr1));

}
}

```

Output:



```

BlueJ: Terminal Window - Kritin11
Options

Arr1[10, 20, 30, 40, 50]

Months: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec

Anonymous Array[]:
[100, 200, 300, 400]

Arr1: [10, 20, 30, 40, 50]

```

ArrayIndexOutOfBoundsException

In Java, the `ArrayIndexOutOfBoundsException` is a runtime exception. Thrown by the Java Virtual Machine (JVM) when attempting to access an invalid index of an array. This exception occurs if the index is negative, Equal to the size of the array or greater than the size of the array while traversing the array.

Homework:

1: Ordered Insertion in an Array (Without Sorting)

Write a program in Java to input N integers into an array in ascending order without using any sorting algorithm.

The user may enter the numbers in any order. As each number is entered, insert it into its correct position in the array by shifting the larger elements one position to the right.

Display the array after every insertion and finally print the complete sorted array.

Sample Input

Enter number of elements: 6

Enter number: 20

Enter number: 10

Enter number: 15

Enter number: 5

Enter number: 18

Enter number: 12

Output

After inserting 20

[20]

After inserting 10

[10, 20]

After inserting 15

[10, 15, 20]

After inserting 5

[5, 10, 15, 20]

After inserting 18

[5, 10, 15, 18, 20]

After inserting 12

[5, 10, 12, 15, 18, 20]

Restrictions

- Do not use Arrays.sort().
- Do not use any Collection classes.
- Insert each element at its proper position while taking input.

2: Second Highest Distinct Element

Write a program to input N integers into an array and determine the second highest distinct element without sorting the array.

If the second highest distinct element does not exist, display an appropriate message.

Sample Input 1

8

15 22 11 45 45 10 39 22

Output

Highest = 45

Second Highest = 39

Sample Input 2

5

20 20 20 20 20

Output

Second Highest Element does not exist.

Restrictions

- Do not sort the array.
- Traverse the array only once after input.

- Do not use an additional array.

3: Insert an Element at a Given Position

Write a program to create an array of N elements.

Input a new element X and a position P (0-based index). Insert the element at the specified position by shifting the remaining elements to the right.

If the array is already full, display "Insertion not possible."

Example

Original Array

[10, 20, 30, 40, 50]

Enter element : 25

Enter position : 2

Output

Array after insertion

[10, 20, 25, 30, 40, 50]

Restrictions

- Do not create another array.
- Perform insertion using shifting only.

4: Maximum Difference

Write a program to input N integers into an array.

Find the largest positive difference between any two elements such that the larger element appears after the smaller element.

If no such pair exists, print 0.

Example

Input

9

7 1 5 3 6 4 8 2 10

Output

Largest Difference = 9

(10 - 1)

Example

Input

5

10 9 8 7 6

Output

Largest Difference = 0

Restrictions

- Do not sort the array.
- Use only one traversal after taking input.

5: Array Rotation Challenge

Write a program to input N integers into an array.

Input an integer K and rotate the array to the right by K positions.

If K is greater than N, rotate by the effective number of positions.

Example

Input

Array

10 20 30 40 50 60

K = 2

Output

50 60 10 20 30 40

Restrictions

- Do not use another array of size N.
- Perform the rotation manually.

6: Longest Consecutive Increasing Sequence

Write a program to input N integers into an array.

Determine the longest consecutive increasing sequence present in the array.

Print

- Starting index
- Ending index
- Length of the sequence
- Elements of the sequence

Example

Input

12

5 8 10 15 3 7 9 11 13 2 4 6

Output

Longest Consecutive Increasing Sequence

7 9 11 13

Starting Index = 5

Ending Index = 8

Length = 4

Example

Input

8

9 8 7 6 5 4 3 2

Output

Longest Consecutive Increasing Sequence

9

Length = 1

Restrictions

- Do not sort the array.
- Do not use any additional array.
- Solve using a single traversal.

7: Smallest Missing Positive Integer

Write a program in Java to input N integers into an array. The array may contain positive numbers, negative numbers, and zero.

Determine the smallest positive integer that is not present in the array.

Sample Input 1

Enter number of elements: 8

3 4 -1 1 7 2 6 8

Output

Smallest Missing Positive Integer = 5

Sample Input 2

Enter number of elements: 5

1 2 3 4 5

Output

Smallest Missing Positive Integer = 6

Restrictions

- Do not sort the array.
- Do not use any Collection classes (ArrayList, HashSet, etc.).
- Do not use library search methods.
- Solve the problem using array traversal and logical reasoning.

8: Leaders in an Array

An element in an array is called a Leader if it is greater than every element

to its right. The last element of the array is always considered a leader. Write a Java program to input N integers into an array and display all the leader elements in the order in which they appear.

Sample Input

Enter number of elements: 8

16 17 4 3 5 2 8 1

Output

Leader Elements are:

17

8

1

Sample Input

Enter number of elements: 6

30 25 20 15 10 5

Output

Leader Elements are:

30

25

20

15

10

5

Restrictions

- Do not sort the array.
- Do not use another array to store the leaders.
- Display the leaders in their original order of appearance.
- Aim to solve the problem with a single traversal from right to left.

9. Without sorting,

Write a Program to create an array of size N, and find the second highest and second lowest value from it.

What are Multi-Dimensional Arrays in Java?

Multi-dimensional array is array of arrays, where each element of the main array holds another array. They are commonly used to represent data in table or matrix formats such as grids, charts or spreadsheets.

Multi-dimensional array can be compared to a table. For example, storing marks of students where each row represents the student and each column represents the subject. In this way data is organized in rows and columns using a multi-dimensional array.

Two-dimensional (2D) Array

A two dimensional array (2D) array is a type of multi-dimensional array that stores the data in the form of rows and columns. It is used to represent table-like structures such as matrices, grids or tables where data is organized in two dimensional only.

Declaration

You can use one of the following syntaxes to declare two dimensional array.

Syntax:

```
DataType[][] arrayname;
```

Or

```
DataType arrayName[][];
```

For memory allocation:

```
arrayName = new datatype[numberOfRows][numberOfColumns];
```

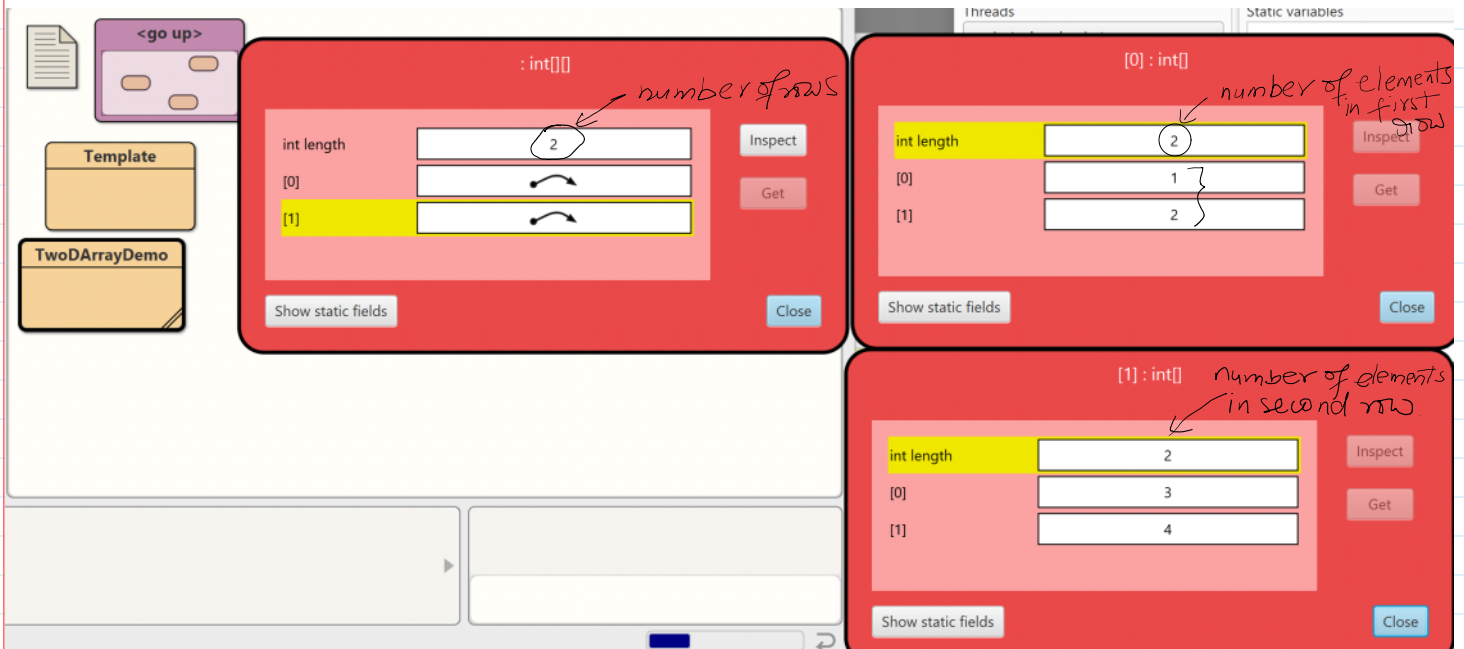
Or

Declaration along with memory allocation:

```
Datatype[][] arrayName = new datatype[numberOfRows][numberOfColumns];
```

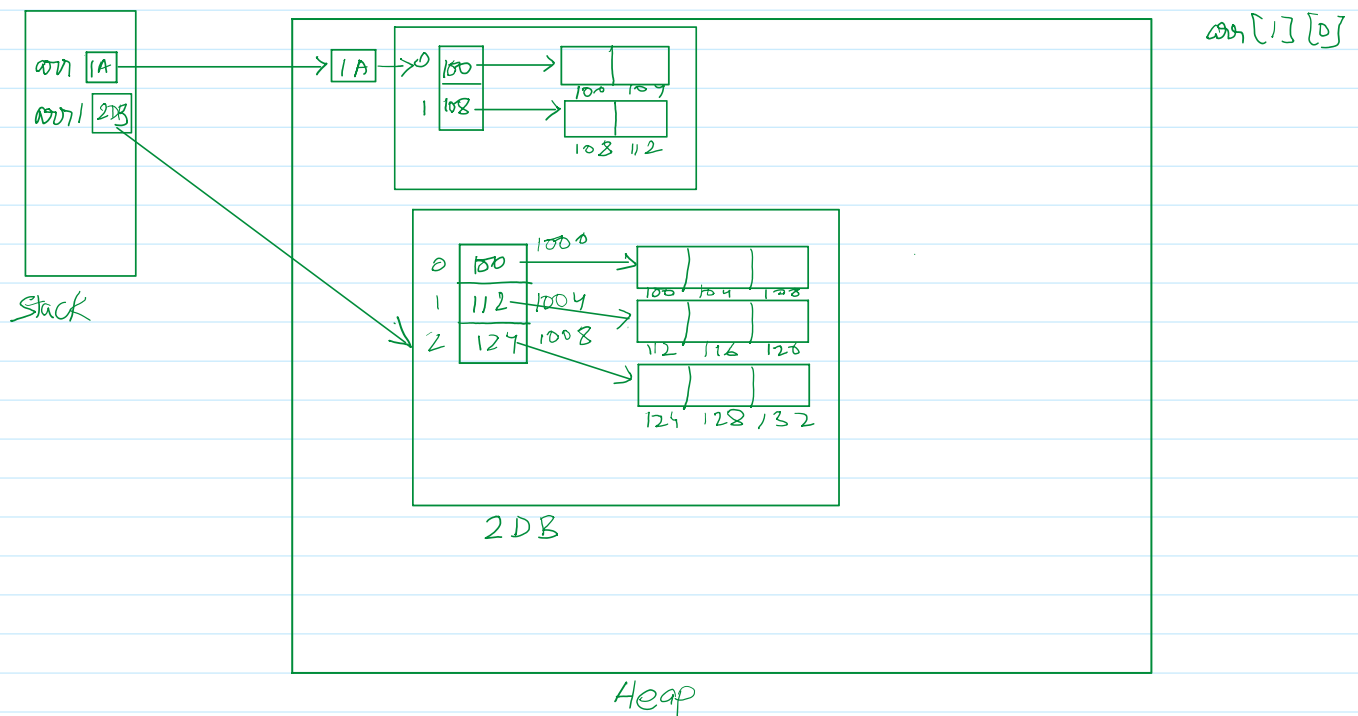
Or

```
Datatype arrayName[][] = new datatype[numberOfRows][numberOfColumns];
```



memory of 2D array is allocated by 'new' keyword dynamically during the execution of code, on the heap memory area.

default values assigned by JVM.



Default values assigned by Java compiler in array:

- For integer type array: 0 (byte, short, int, long)
- For floating point type: 0.0 (float, double)
- For boolean: false
- For char: '\u0000' or null
- For String: null

For long type array:

The screenshot shows the BlueJ IDE with a Java program named `TwoArrayDemo`. The code defines a 3x3 long array `arr2` and initializes it with zeros. The debugger is open, showing the state of the array at different indices:

- Array State:**
 - `arr2` (length 3): [0] = 0, [1] = 0, [2] = 0
 - `arr2[0]` (length 3): [0] = 0, [1] = 0, [2] = 0
 - `arr2[1]` (length 3): [0] = 0, [1] = 0, [2] = 0
 - `arr2[2]` (length 3): [0] = 0, [1] = 0, [2] = 0
- Local Variables:**
 - `String[] args` = <object>
 - `int[] arr` = <object>
 - `int[] arr1` = <object>
 - `long[] arr2` = <object>
 - `char[] arr3` = <object>
 - `String[] arr4` = <object>
 - `float[] arr5` = <object>
 - `double[] arr6` = <object>
 - `boolean[] arr7` = <object>

For char[][] type:

The screenshot shows the BlueJ IDE with the same Java program. The code defines a 3x3 char array `arr3` and initializes it with '\u0000'. The debugger is open, showing the state of the array at different indices:

- Array State:**
 - `arr3` (length 3): [0] = '\u0000', [1] = '\u0000', [2] = '\u0000'
 - `arr3[0]` (length 3): [0] = '\u0000', [1] = '\u0000', [2] = '\u0000'
 - `arr3[1]` (length 3): [0] = '\u0000', [1] = '\u0000', [2] = '\u0000'
 - `arr3[2]` (length 3): [0] = '\u0000', [1] = '\u0000', [2] = '\u0000'
- Local Variables:**
 - `String[] args` = <object>
 - `int[] arr` = <object>
 - `int[] arr1` = <object>
 - `long[] arr2` = <object>
 - `char[] arr3` = <object>
 - `String[] arr4` = <object>
 - `float[] arr5` = <object>
 - `double[] arr6` = <object>
 - `boolean[] arr7` = <object>

Default values for String[][]:

```

7 public class TwoArrayDemo
8 {
9     public static Scanner sc = new Scanner(System.in);
10    public static void main(String[] args){
11        int[][] arr = {{1,2}, {3, 4}};
12
13        int[][] arr1 = new int[3][3];
14        long[][] arr2 = new long[3][3];
15        char[][] arr3 = new char[3][3];
16        String[][] arr4 = new String[3][3];
17
18        float[][] arr5 = new float[3][3];
19        double[][] arr6 = new double[3][3];
20
21        boolean[][] arr7 = new boolean[3][3];
22    }
23 }

```

Class compiled - no syntax errors

Static variables: public Scanner sc = ...

Instance variables: (empty)

Local variables: String[] args = ...; int[][] arr = ...; int[][] arr1 = ...; long[][] arr2 = ...; char[][] arr3 = ...; String[][] arr4 = ...; float[][] arr5 = ...; double[][] arr6 = ...; boolean[][] arr7 = ...

Default values for float[][] or double[][] is same:

```

7 public class TwoArrayDemo
8 {
9     public static Scanner sc = new Scanner(System.in);
10    public static void main(String[] args){
11        int[][] arr = {{1,2}, {3, 4}};
12
13        int[][] arr1 = new int[3][3];
14        long[][] arr2 = new long[3][3];
15        char[][] arr3 = new char[3][3];
16        String[][] arr4 = new String[3][3];
17
18        float[][] arr5 = new float[3][3];
19        double[][] arr6 = new double[3][3];
20
21        boolean[][] arr7 = new boolean[3][3];
22    }
23 }

```

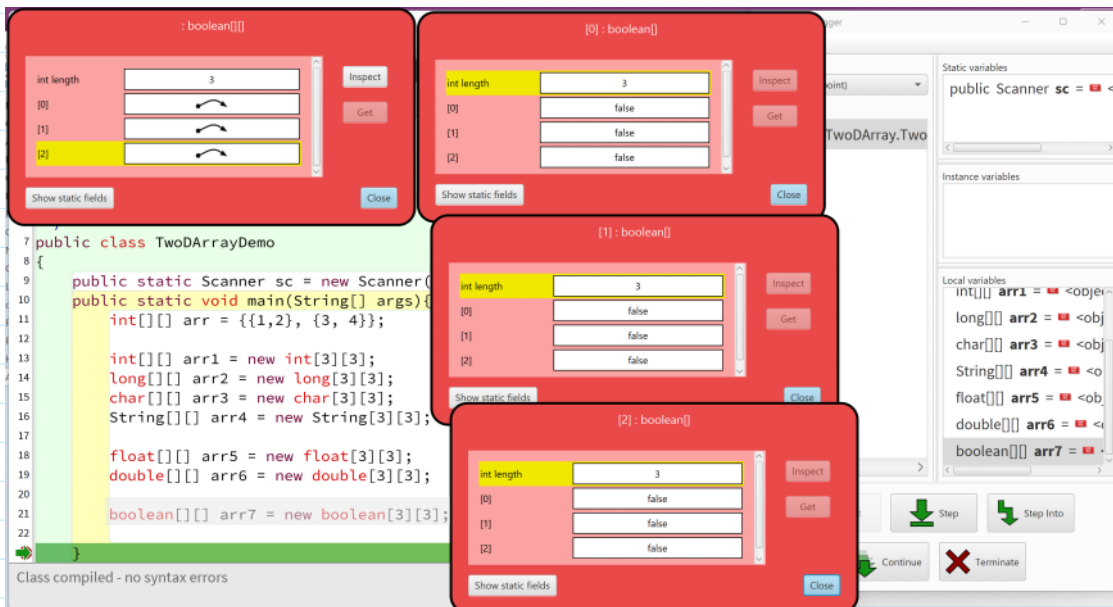
Class compiled - no syntax errors

Static variables: public Scanner sc = ...

Instance variables: (empty)

Local variables: int[][] arr1 = ...; long[][] arr2 = ...; char[][] arr3 = ...; String[][] arr4 = ...; float[][] arr5 = ...; double[][] arr6 = ...; boolean[][] arr7 = ...

Default values for boolean[][] is false:



The representation of 2D array

The following image shows the representation of 2D array.

columns
↓ ↓ ↓
0 1 2

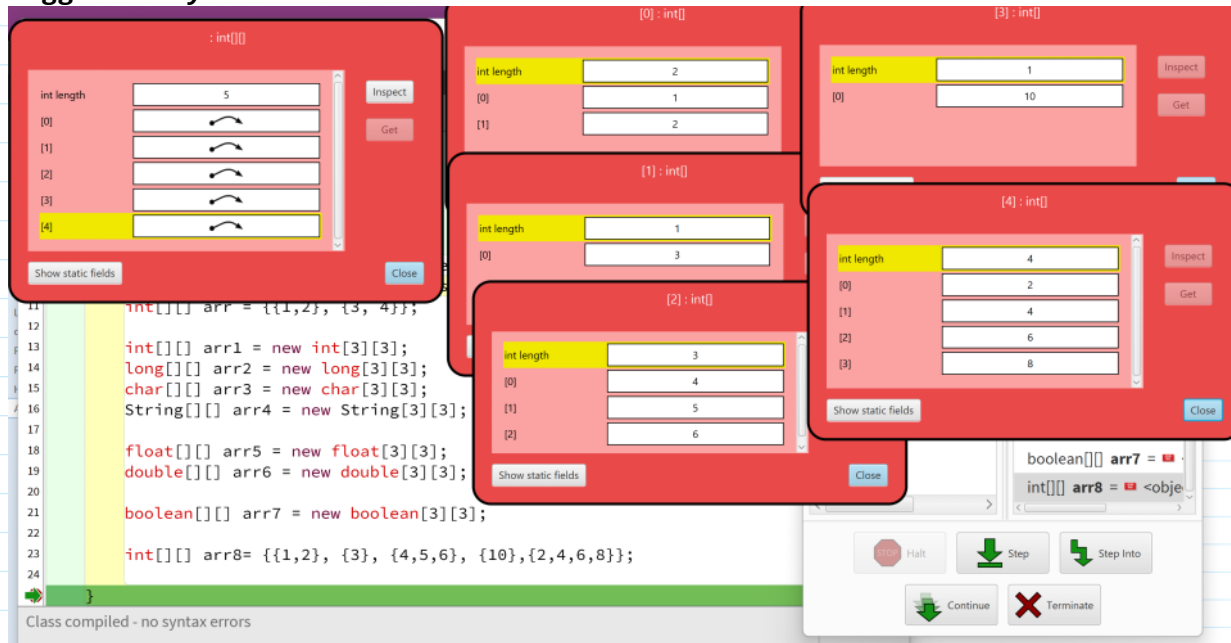
rows
→ 0
→ 1
→ 2

0,0	0,1	0,2
1,0	1,1	1,2
2,0	2,1	2,2

Two dimensional array in memory:

0,0 0,1 0,2 1,0 1,1 1,2 2,0 2,1 2,2

Jagged arrays



```

int[][] arr8= {{1,2}, {3}, {4,5,6}, {10},{2,4,6,8}};
int i, j;
System.out.print("\fJagged Array:\n");
for(i = 0; i < arr8.length; i++, System.out.print("\n")){
    for(j = 0; j < arr8[i].length; j++){
        System.out.print("\t" + arr8[i][j]);
    }
}

```

number of rows (pointing to `arr8.length`)

number of columns (pointing to `arr8[i].length`)

Output:

```

BlueJ: Terminal Window - Kritin11
Options
Jagged Array:
1      2
3
4      5      6
10
2      4      6      8

```

```

package MyArrays.TwoDArray;
import java.util.*;
/**
 * @author (your name)
 * @version (a version number or a date)
 */
public class TwoDArrayDemo
{
    public static Scanner sc = new Scanner(System.in);
    public static void main(String[] args){
        int[][] arr = {{1,2}, {3, 4}};

        int[][] arr1 = new int[3][3];
        long[][] arr2 = new long[3][3];
        char[][] arr3 = new char[3][3];
        String[][] arr4 = new String[3][3];

        float[][] arr5 = new float[3][3];
        double[][] arr6 = new double[3][3];

        boolean[][] arr7 = new boolean[3][3];

        int[][] arr8= {{1,2}, {3}, {4,5,6}, {10},{2,4,6,8}};
        int i, j;
        System.out.print("\fJagged Array:\n");
        System.out.print("\nTechnique-1\n");
        for(i = 0; i < arr8.length; i++, System.out.print("\n")){
            for(j = 0; j < arr8[i].length; j++){
                System.out.print("\t" + arr8[i][j]);
            }
        }
    }
}

```

```

System.out.print("\nTechnique-2\n");
//For each loop only applicable for Collection (array list. ...)
for(int[] x: arr8){
    for(int y : x){
        System.out.print("\t"+y);
    }
    System.out.print("\n");
}

System.out.print("\nTechnique-3\n");
//For each loop only applicable for Collection (array list. ...)
for(int[] x: arr8){
    for(i=0; i < x.length; i++){
        System.out.print("\t"+x[i]);
    }
    System.out.print("\n");
}

System.out.print("\nTechnique-4");
//For each loop only applicable for Collection (array list. ...)
for(int[] x: arr8){
    System.out.print("\n"+java.util.Arrays.toString(x));
}

System.out.print("\nTechnique-5");
for(i = 0; i < arr8.length; i++){
    System.out.print("\n"+java.util.Arrays.toString(arr8[i]));
}
}
}

```

Output:

Jagged Array:

Technique-1

```

1  2
3
4  5  6
10
2  4  6  8

```

Technique-2

```

1  2
3
4  5  6
10
2  4  6  8

```

Technique-3

```

1  2
3
4  5  6
10
2  4  6  8

```

Technique-4

[1, 2]
[3]
[4, 5, 6]
[10]
[2, 4, 6, 8]

Technique-5

[1, 2]
[3]
[4, 5, 6]
[10]
[2, 4, 6, 8]

Three Dimensional (3D) Array (Not in ISC)

A 3 dimensional (3D) array is an array that contains arrays of arrays. It is used to store data in three dimensions such as layers, rows and columns. A 3D array is commonly used to represent complex data structures like a collection of matrices or data arranged across multiple levels.

It is a complex form of 2D array. In other words, it is an array of two dimensional array.

Declaration

Syntax:

`DataType[][][] arrayName;`

Or

`DataType arrayName[][][];`

For memory allocation:

`arrayName = new Datatype[layers][numberOfRows][numberOfColumns];`

Or

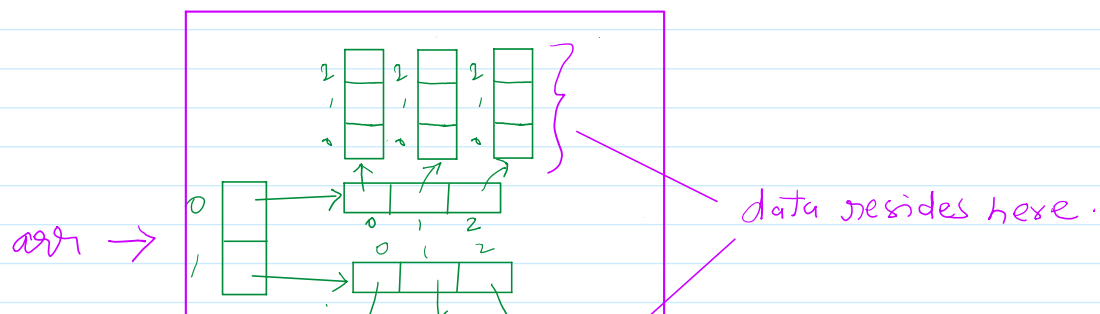
Declaration along with memory allocation

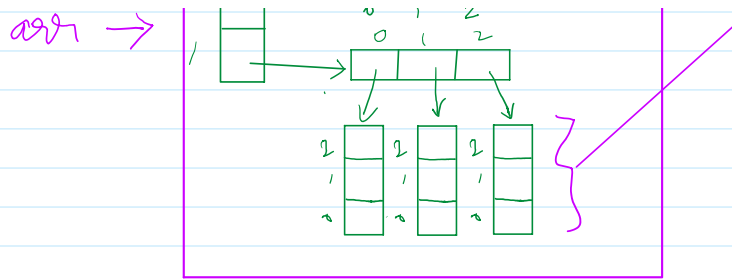
`DataType[][][] arrayName = new DataType[layers][numberOfRows][numberOfColumns];`

Or

`DataType arrayName[][][] = new DataType[layers][numberOfRows][numberOfColumns];`

`int arr[][][] = new int[2][3][3];`





Three dimensional Array.

```
package MyArrays.TwoDArray;
import java.util.*;
```

```
/**
 * Write a description of class ThreeArrayDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ThreeArrayDemo
{
    public static void main(String[] args){
        System.out.print("\f");
        Random r = new Random();
        int[][][] arr = new int[2][3][3];
        int i, j, k;
        for(i = 0; i < arr.length; i++){
            System.out.print("\nLayer-"+i+"\n");
            for(j = 0; j < arr[i].length; j++, System.out.print("\n")){
                for(k = 0; k < arr[i][j].length; k++){
                    arr[i][j][k] = r.nextInt(10, 100);
                    System.out.print("\t" + arr[i][j][k] );
                }
            }
        }
    }
}
```

Output:

BlueJ: Terminal Window - Kritin11			
Options			
Layer-0			
70	48	55	
45	37	60	
94	76	96	
Layer-1			
56	95	53	
93	81	55	
60	94	41	

Notes In Java array is the template of Arrays class present in java.util package. Therefore, internally it assigns number of elements in the length variable, which is public instance variable in Arrays class.

array.length
↑

directly accessible because it is a public int variable in Arrays class.

